

Dependency-Frontier Scheduling with Asymmetric-Cost Spill Planning for NPU Kernels

Chengzhi Gao*
School of Mechanical Engineering,
Southeast University
Jiangning District, Nanjing, China
contact@vennai.org

Jun Huang
School of Mechanical Engineering,
Southeast University
Jiangning District, Nanjing, China
hj992881627@outlook.com

Qin Ye
School of Mechanical Engineering,
Southeast University
Jiangning District, Nanjing, China
yq020319@163.com

Abstract

NPU kernel scheduling must jointly choose a legal topological order of a micro-operation DAG, contiguous on-chip buffer addresses, and a spill plan when capacity is insufficient. We study the evaluator's static asymmetric cost: interrupting an input-backed buffer costs one transfer, whereas interrupting a generated, unbacked buffer costs two, so $P2$ traffic admits the auditable decomposition $Tr = Vol + Dt$. We present a scalable bounded portfolio that combines a dependency-frontier order with three existing orders, best-fit placement, and two eviction heuristics, selecting complete artifacts by the $P2$ traffic or $P3$ time objective. For a fixed topological order, we formulate a weighted residency-gap CP-SAT relaxation whose optimum is a traffic lower bound; a validated contiguous packing that attains it gives a fixed-order traffic certificate. On six public DAGs, the solver has five $P2$ wins and one tie (median reduction 0.866%, maximum 9.08%), and five $P3$ wins with one slowdown (median improvement 3.77%, sole regression 4.23%). Three fixed-order certificates cover two public instances. Conv_Case0 has certified optima of 57,408 bytes under dependency-frontier and 81,504 under the legacy $P1$ order, a 29.6% order-induced difference. Two nonuniform repair case studies and internal synthetic experiments provide bounded mechanism, non-regression, and small-scale same-order evidence only; they are neither a uniform production algorithm nor a new generalization result.

Keywords: kernel scheduling; neural network processor; spill planning; scratchpad memory; compiler optimization

*Corresponding author. Co-Founder, Vennai.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CGO '27,

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

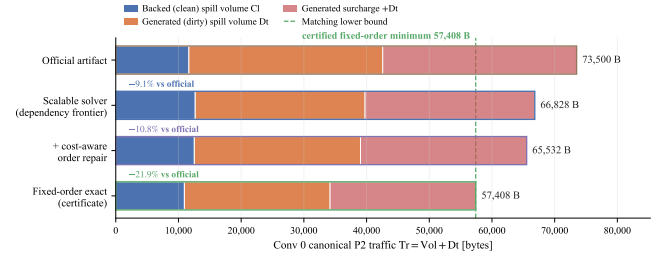


Figure 1. The exact-to-heuristic evidence ladder on Conv_Case0, decomposed by $Tr = Vol + Dt$. The scalable production solver improves the official artifact by 9.1%; exploratory order repair adds 1.9%; and the weighted-gap planner attains its lower bound under fixed dependency-frontier, certifying 57,408 bytes.

NPU compilers lower each kernel into thousands of micro-operations executing on heterogeneous pipelines and exchanging data through software-managed on-chip memories. Scheduling commits simultaneously to a legal topological order of the micro-operation DAG, contiguous physical addresses for resident buffers, and a spill plan when capacity is insufficient. Allocation, use, and release timing determines both how many bytes leave the chip and which values are evicted; logical live peaks alone capture neither fragmentation, next-use distance, nor asymmetric spill cost.

The benchmark charges one transfer for input-backed buffers and two for generated, unbacked buffers. If Vol is total spilled volume and Dt its generated part, then $Tr = Vol + Dt$. Traffic changes must therefore be explained through both total volume and the generated surcharge. Section 2 gives the formal definition and tight bounds; Supplementary Sec. 2.1 gives the complete proof.

We separate topological-order construction from physical spill planning and use a machine-checkable exact planner to quantify scalable-heuristic headroom (Figure 1). The production solver uses a *dependency-frontier* order to prevent starvation of ready predecessor groups, then selects by the true $P2/P3$ objective among a small set of structural orders, placement/eviction policies, and reload windows. A

fixed-order weighted residency-gap oracle supplies a traffic lower bound; validated contiguous packing that attains the bound yields a fixed-order traffic certificate. Offline repair and exact planning are reported as research tools, not default stages. The complete solution process appears in Supplementary Sec. 3.

Our contributions are: (1) a scalable dependency-frontier scheduler integrated with a true-objective spill-planning portfolio; (2) a weighted residency-gap lower bound that upgrades to a fixed-order certificate through a validated contiguous artifact; and (3) an audit-oriented evaluation organized around $Tr = Vol + Dt$ and a strict separation of production, repair case studies, and the exact oracle. On six public instances, production has five P2 wins and one tie and five P3 time wins with one loss. The Conv_Case0 oracle certifies 57,408 bytes and exposes a 29.6% fixed-order difference from the legacy P1 order. Nonuniform repair and internal synthetic results are additional bounded evidence only; see Section 4 and Supplementary Sec. 4.

The source code and reproduction guide are available online.¹

2 Problem Model

The input is a micro-operation DAG $G = (\mathcal{V}, \mathcal{E})$. Operation nodes carry a pipeline, cycle count, and buffer set; cache-management nodes give each buffer b a unique allocation a_b , free f_b , size s_b , and cache type $\tau(b)$. A base schedule is a topological order of all original nodes. Evaluation uses $L1 = 4096$, $UB = 1024$, $L0A = 256$, $L0B = 256$, and $L0C = 512$.

The evaluator assigns a *static* class: if b appears in any COPY-IN, then $\kappa_b = 1$ (backed); otherwise $\kappa_b = 2$ (generated and unbacked). One spill episode costs $tr_b = \kappa_b s_b$. Since the input does not expose per-operation read/write roles, this is an evaluator definition, not dynamic dirty-state inference. Let backed and generated spill volumes be Cl, Dt :

$$Tr = Cl + 2Dt = Vol + Dt, \quad Vol = Cl + Dt. \quad (1)$$

Thus $Vol \leq Tr \leq 2Vol$. At fixed volume, class mix changes traffic by at most two, while ordering and placement can also change Vol ; tightness and scope appear in Supplementary Sec. 2.1.

A physical residency segment occupies a contiguous interval in its cache, and simultaneous segments may not overlap. A SPILL-OUT/SPILL-IN pair interrupts residency, possibly reloading at another address. Original users and FREE are mandatory-residency events. P1 scores logical ALLOC-FREE peaks only. P2 emits the augmented schedule, addresses, and spill table and uses (Tr, n, T) ; P3 uses (T, Tr, n) , where T is pipelined earliest completion. P1 logical peaks are not P2 physical peaks, and P3 may accept traffic to reduce time, so the views are never mixed. Every

physical result is checked for coverage, topology, contiguous packing, and spill dependencies. The complete evaluator contract and paired P2-P3 tradeoff are in Supplementary Secs. 1 and 4.4.

3 Method

We separate three roles with different evidence status. The *production solver* is a bounded candidate-order $\times$ online spill-planner portfolio and the only default component. *Cost-aware repair* is an offline case study of whether cost semantics can improve an order. The *fixed-order exact planner* is a CP-SAT research oracle for lower bounds and certificates.

3.1 Production Solver

Dependency-frontier ordering. Many simultaneously ready allocations can let a one-input stream starve a multi-input consumer and prolong its prepared operands. Dependency-frontier schedules frees and executable operations first; when only allocations remain, it completes the ready predecessor group of the earliest unlockable consumer (Figure 3, Algorithm 1). The new frontier order uses DAG structure and identifiers only. The portfolio also retains capfit-id, the P1 order, and id-raw; we do not relabel the whole portfolio as cost-aware or motif-agnostic. Complete tie rules are in Supplementary Sec. 3.1.

Algorithm 1 Canonical scalable solver

Require: instance I , task $p \in \{1, 2, 3\}$

```

1: if  $p = 1$  then
2:   return P1MEMORYORDER( $I$ )
3: end if
4:  $\mathcal{O} \leftarrow \{\text{FRONTIER}(I), \text{CAPFITID}(I)\}$ 
5: Add P1ORDER( $I$ ) and IDRAW( $I$ ) to  $\mathcal{O}$ 
6:  $\mathcal{R} \leftarrow \{\text{DISTCOST}, \text{ADAPTIVE25}\}$ 
7:  $\mathcal{H} \leftarrow \{0, 5, 40\}$  if  $p = 2$ , else  $\{0, 5, 40, 80, 120\}$ 
8:  $A^* \leftarrow \perp$ ;  $k^* \leftarrow (+\infty, +\infty, +\infty)$ 
9: for all  $(S, r, H) \in \mathcal{O} \times \mathcal{R} \times \mathcal{H}$  do
10:    $A \leftarrow \text{BESTFITPLAN}(I, S, r, H)$ 
11:   if  $A$  is feasible then
12:      $(Tr, n, T) \leftarrow \text{EVALUATE}(A)$ 
13:      $k \leftarrow (Tr, n, T)$  if  $p = 2$ , else  $(T, Tr, n)$ 
14:     if  $k < k^*$  then
15:        $(A^*, k^*) \leftarrow (A, k)$ 
16:     end if
17:   end if
18: end for
19: return  $A^*$  ▷ order, offsets, and spill entries

```

Placement and eviction. Every base order uses true best-fit contiguous placement and compares a distance/cost score with an adaptive fragmentation policy. Both are

¹<https://github.com/VennIntelligence/2025A-kernel-sched>

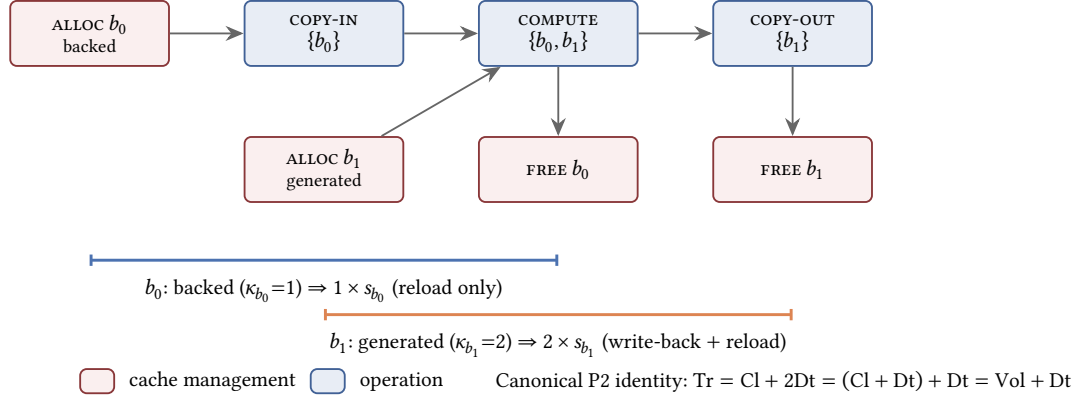


Figure 2. Micro-operation DAG and benchmark spill classes. Cache-management nodes delimit logical lifetimes. Input-backed buffers incur one transfer per interruption; generated, unbacked buffers incur two.

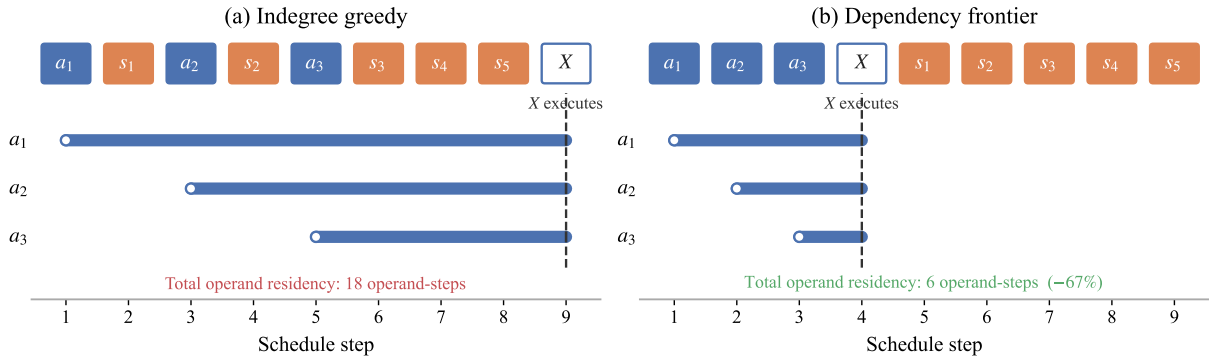


Figure 3. Predecessor-group completion. An indegree-greedy order delays the three-operand consumer X to step 9 and accumulates 18 operand-steps; frontier fires X at step 4 and reduces residency to 6 operand-steps (−67%).

heuristics; true-objective selection over complete artifacts absorbs local mistakes. Scores, thresholds, relocation bounds, and tie rules are in Supplementary Sec. 3.1 and Table S2.

True-objective portfolio selection. P2/P3 form fixed portfolios over four orders, two policies, and three/five reload windows, selecting complete artifacts by (Tr, n, T) or (T, Tr, n) . Infeasible members are skipped; no live-byte or overflow surrogate selects the output. Window semantics and artifact generation are in Supplementary Secs. 3.1–3.3.

3.2 Cost-Aware Repair Case Studies

To probe order-level cost headroom, we try legal node or chain moves offline, replan, and accept only strict P2-key improvements. Different instances use different search procedures and budgets; small FlashAttention probes find no gain and Matmul is not searched. Repair is therefore neither a production stage nor a uniform algorithm. Budgets and negative probes appear in Supplementary Secs. 3.4 and 4.5.

3.3 Fixed-Order Weighted Residency Gaps

Under a fixed order, consecutive mandatory events of each buffer define residency gaps. Keeping a gap consumes capacity; interrupting it pays one class-weighted charge. A binary model minimizes interruption cost under per-cache, per-cut capacity. Its solution must still undergo contiguous packing, spill-dependency generation, and canonical validation (Figure 4). Variables, cuts, the contiguity counterexample, and model encoding are in Supplementary Secs. 2.2–2.3 and 3.4.

Proposition 3.1 (Fixed-order traffic lower bound and certificate). *For a fixed order, the relaxation optimum lower-bounds every legal contiguous-address spill plan. If an optimal gap selection packs contiguously and its validated artifact has traffic equal to the bound, the artifact is traffic-optimal for that fixed order under the benchmark model.*

The proof maps any legal physical plan to a capacity-feasible gap selection. The complete proof and a capacity-feasible but unpackable counterexample are in Supplementary Secs. 2.2–2.3. The certificate optimizes fixed-order traffic only, not spill count or time among ties, and proves no global order optimum.

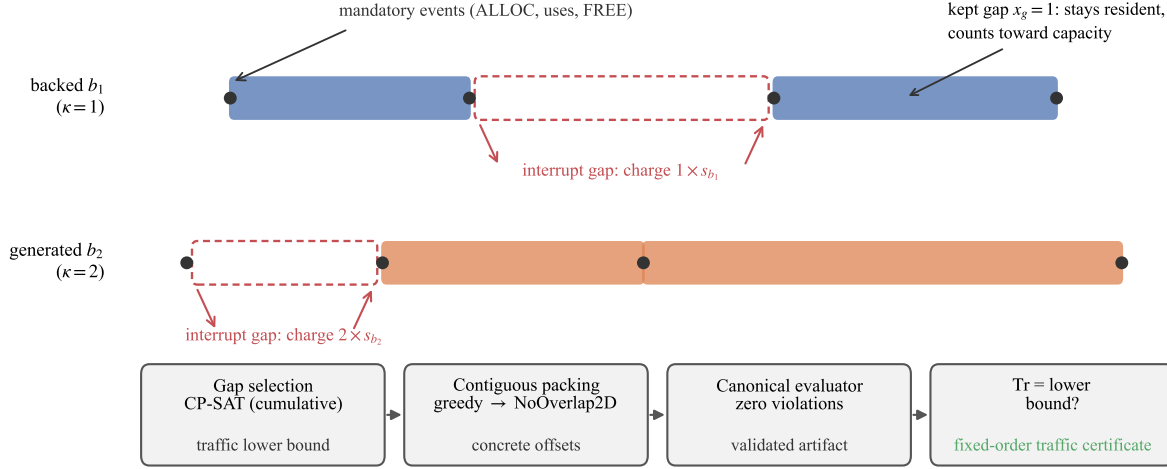


Figure 4. Weighted residency gaps. Kept gaps consume capacity; interrupted gaps pay $\kappa_b s_b$. CP-SAT supplies a lower bound, while independent contiguous packing and canonical validation upgrade a bound-attaining artifact to a fixed-order traffic certificate.

Complexity and deployment. With fixed candidate counts, the scalable path is polynomial with a conservative quadratic buffer term in placement; gap selection and fallback packing are exponential in the worst case. CP-SAT remains a research oracle. Any integration requires a threshold/timeout and validated fallback; details are in Supplementary Sec. 2.5.

4 Evaluation

The six public DAGs are the released instances of Problem A, “Intra-kernel Scheduling on a General-Purpose Neural Network Processor,” from the 2025 Huawei Cup China Graduate Mathematical Contest in Modeling. They cover convolution, FlashAttention, and matrix multiplication and range from 1,716 to 36,086 nodes. All experiments use the capacities and evaluator of Section 2; every production artifact has zero violations. Official artifacts are used only for final comparison, and no order generator reads an official schedule. The scalable solver is the sole production default; repair and exact planning are reported separately.

4.1 Public P2 and P3 Results

Table 1 and Figure 5 give the primary official-artifact comparison. P2 has five strict wins and one tie (median reduction 0.866%, maximum 9.08%); P3 time has five wins and one loss (median improvement 3.77%), with Conv_1 the sole 4.23% regression. Complete P1/P2/P3 metrics, counts, tie-break times, and validation states are in Supplementary Tables S5–S7.

A controlled $H = 0$ ablation is descriptively nonadditive but is not a complete factorial design. Conv_1 remains 214 bytes above the actual predecessor, and both Matmul cases

Table 1. Canonical production solver versus official P2/P3 artifacts.

Instance	P2 ours	P2 official	P3 time: ours/official
Conv_0	66,828	73,500	515,634/535,312
Conv_1	72,734	73,240	1,118,687/ 1,073,322
FA_0	3,584	3,692	36,344/46,761
FA_1	32,512	32,840	152,364/193,059
Matmul_0	34,688	34,944	186,820/194,331
Matmul_1	460,800	460,800	1,771,383/1,800,218

are unchanged. We therefore interpret complete configurations, not isolated main effects or synergy; Supplementary Table S8 and Figure S1 give the full path.

4.2 Tr = Vol + Dt Accounting

The cases span generated-only, mixed, and backed-only regimes (Figure 7). Every strict P2 win reduces Vol; Conv_1 wins despite increasing Dt by 171 bytes, while Matmul_0 wins with Dt = 0. No single composition story explains the results. Per-case accounting is in Supplementary Table S9 and Figure S2.

Two nonuniform repair studies hold the planner fixed and change only a legal order, reducing Conv_0/Conv_1 by 1,296/1,794 bytes. Post-hoc inspection associates accepted moves with advancing backed ALLOC/COPY-IN chains, but Vol and Dt change together, so this does not isolate a cause. Heterogeneous budgets, negative FlashAttention probes, and untested Matmul appear in Supplementary Table S11.

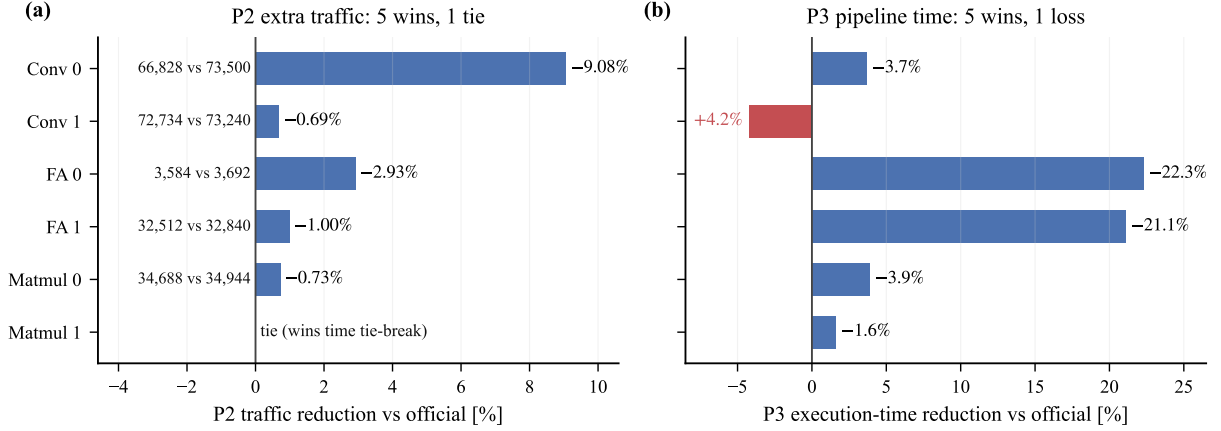


Figure 5. Production results on all six public instances: P2 traffic reduction against official artifacts and P3 execution-time reduction. Repair and the fixed-order oracle are excluded.

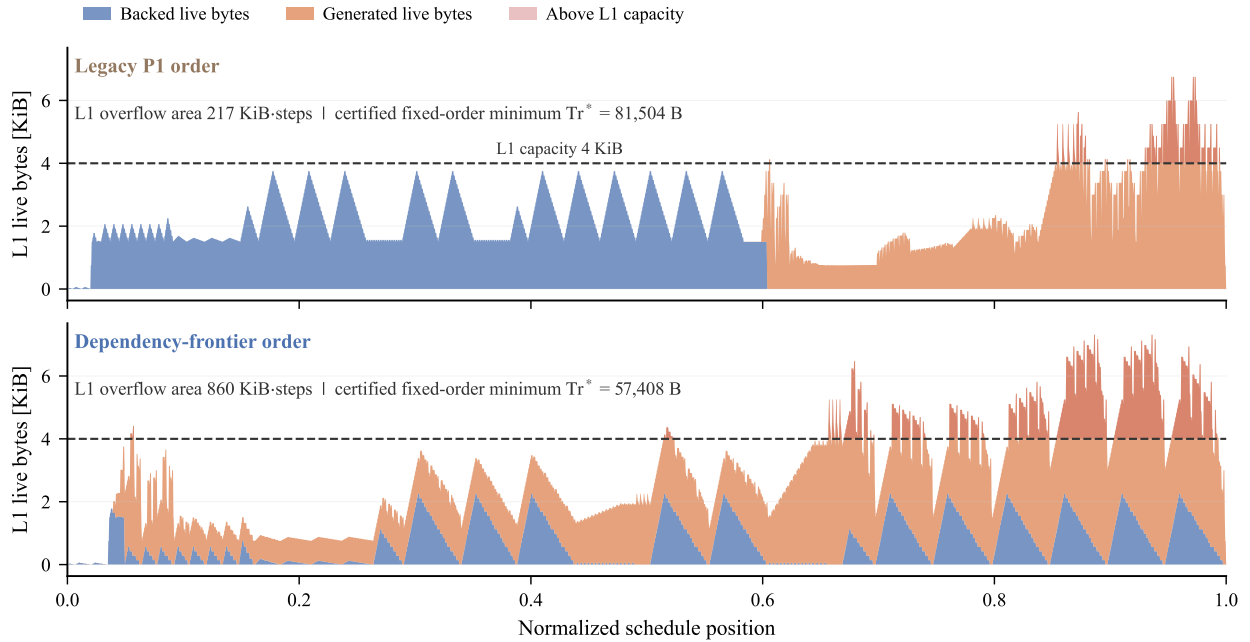


Figure 6. Logical L1 residency on Conv_Case0 under two certified topological orders, decomposed into backed and generated live bytes. The dependency-frontier order has a larger geometric overflow area but a 29.6% lower certified fixed-order traffic optimum (57,408 versus 81,504 bytes). Geometric pressure is therefore not a reliable order-ranking surrogate under asymmetric cost.

4.3 Exact-to-Heuristic Evidence

The Conv_0/frontier fixed-order certificate is 57,408 bytes, 14.1% below the scalable plan under that order. The same oracle certifies 81,504 bytes under the legacy P1 order, isolating a 29.6% order difference. FA_0 supplies the third certificate and covers a second public instance. Other cases have positive gaps, timeout, or were not run (Table 2) and are not traffic-optimal. Certificates do not optimize P2 spill-count

or time tie-breakers. The full chain, failures, and machine-artifact scope are in Supplementary Secs. 3.3–3.5 and Table S12.

Order headroom versus pressure. Conv_0/frontier has larger logical L1 overflow area than the legacy P1 order but 29.6% lower certified traffic. Live/overflow-area surrogates therefore cannot reliably rank orders under asymmetric cost (Figure 6).

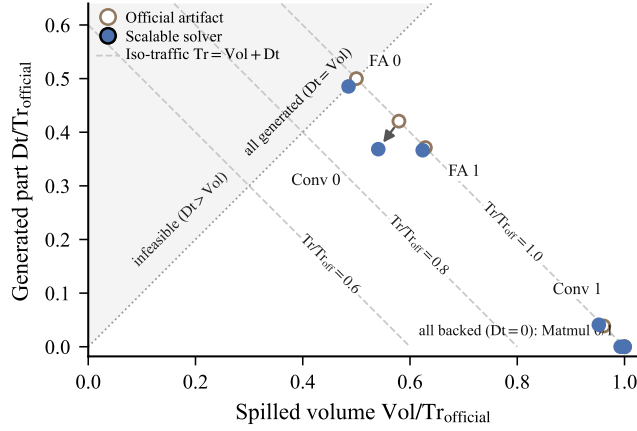


Figure 7. Public cases on the normalized (Vol, Dt) plane. Arrows point from official to scalable artifacts; dashed diagonals are iso-traffic lines. The cases span generated-only, mixed, and backed-only regimes.

Table 2. Repair case studies and fixed-order exact evidence. A dash is no completed result, not zero traffic.

Instance	Scalable	Repair	Fixed-order	Status
Conv_0	66,828	65,532	57,408	Traffic certificate
Conv_1	72,734	70,940	–	Packing timeout
FA_0	3,584	probe	3,584	Traffic certificate
FA_1	32,512	probe	32,512	Feasible; LB 31,936
Matmul_0	34,688	–	34,816	Feasible; LB 29,952
Matmul_1	460,800	–	–	Not run

4.4 Robustness, Negative Results, and Limitations

Capacity sweep. At every scanned feasible capacity point in the Conv_0/L1/fixed- $H = 0$ sweep, the selected configuration never loses to the cp-free-first order comparator; where both traffic values are nonzero, the ratio grows from $1.35\times$ to $124\times$. This is a controlled single-instance, single-cache boundary, not monotonicity of the absolute difference or cross-operator robustness. The full nine-point data, infeasible boundary, and curve are in Supplementary Table S13 and Figure S4.

Internal synthetic boundary. On 36 standard re-evaluations, production ties its predecessor candidate subset; cp-free-first is also identical to included id-row on 36/36 instances, so both zero-loss results are principally portfolio-monotonicity checks. We observe zero losses against four nonincluded fixed implementations; all eight 17-node oracle instances attain the fixed-order traffic optimum under their selected order. This supports non-regression and small-scale same-order agreement, not new generalization over the predecessor. The 36+8 per-instance rows and 252 run-time validations are reported in Supplementary Tables S14–S17; artifact locations and

reproduction commands are maintained in the repository guide.

Scope threats. The public evaluation contains only six DAGs and the exact study is incomplete. The benchmark uses static COPY-IN labels, requires residency at FREE, and exposes no read/write roles, precluding claims about dynamic dirty-state evolution. P1 is a logical footprint, not physical P2 residency; CP-SAT may time out, and scalable victim policies remain heuristic. All claims are limited to the stated evaluator and artifact contract.

5 Related Work

Joint schedule-memory optimization. COSMA [8] jointly optimizes operator scheduling, allocation, and tensor replacement with an ILP, so joint schedule-memory optimization is established prior art and we claim no such first. Our setting is narrower and finer-grained: multi-cache micro-operation DAGs with the benchmark’s static input-backed and generated spill classes, a dependency-frontier production heuristic, and a fixed-order continuous-offset certificate that connects the heuristic to a machine-checked optimum.

Register pressure and rematerialization. Integrated instruction scheduling and register allocation [4, 10, 11] likewise reshapes live ranges under a capacity constraint. Checkmate [6] computes offline rematerialization schedules, and Dynamic Tensor Rematerialization [7] is its online counterpart for dynamic models; both trade recomputation for memory. Our benchmark forbids recomputation and instead demands explicit spill and reload operations with concrete offsets for a static kernel DAG, which is why packing validation is part of our certificate.

Caching and replacement. Belady’s rule for fixed request streams [1] and weighted or variable-size caching [3] already treat nonuniform sizes and retrieval costs, and clean-aware replacement is established in compiler-managed memory [5, 9]. We accordingly make no general weighted-caching claim. The contribution is the concrete combination: a topological request order that the optimizer itself may change, multiple scratchpad caches, canonical one- versus two-transfer spill costs, and a lower-bound-to-physical-packing bridge. Spill-free compilation [2] is complementary; our planner targets instances where the evaluator still requires spill traffic.

6 Conclusion

This work recasts NPU kernel memory planning as two separable questions: which legal order exposes a tractable residency frontier, and which backed or generated gaps should remain resident under capacity. Our production answer is a bounded, exact-objective portfolio built around dependency-frontier ordering; our fixed-order

research answer is a weighted-gap lower bound coupled to a contiguous-packing traffic certificate. The identity $Tr = Vol + Dt$ keeps the source of every P2 change auditable throughout.

The evidence is useful but deliberately narrow. The scalable solver improves five of six public P2 cases and ties the sixth, and improves P3 time on five of six. Nonuniform repair studies help the two convolution cases, and the canonically validated synthetic re-evaluation establishes non-regression and small-scale same-order traffic optimality rather than new generalization. The result is therefore not a universal clean/dirty scheduling principle; it is an honest exact-to-heuristic bridge together with a validated improvement on the studied kernel family. Future work should integrate guarded exact solving behind a timeout with a validated fallback, derive a cheaper and uniform cost-aware order repair, and evaluate on broader hardware traces whose read/write semantics are richer than the current static benchmark labels.

References

- [1] L. A. Belady. 1966. A Study of Replacement Algorithms for a Virtual-Storage Computer. *IBM Systems Journal* 5, 2 (1966), 78–101.
- [2] Prasanth Chatarasi, Alex Gatea, Wei Wang, Chris Bowler, Shubham Jain, Masoud Ataei Jaliseh, Nicole Khoun, Alberto Mannari, Bardia Mahjour, Viji Srinivasan, and Swagath Venkataramani. 2026. Enabling Spill-Free Compilation via Affine-Based Live Range Reduction Optimization. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 1–13.
- [3] Christian Coester, Roie Levin, Joseph (Seffi) Naor, and Ohad Talmon. 2022. Competitive Algorithms for Block-Aware Caching. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [4] James R. Goodman and Wei-Chung Hsu. 1988. Code Scheduling and Register Allocation in Large Basic Blocks. In *Proceedings of the 2nd International Conference on Supercomputing (ICS)*. 442–452.
- [5] Jia Guo, Maria Jesus Garzaran, and David Padua. 2004. The Power of Belady’s Algorithm in Register Allocation for Long Basic Blocks. In *Languages and Compilers for Parallel Computing (LCPC) (Lecture Notes in Computer Science, Vol. 2958)*. Springer, 374–389. doi:10.1007/978-3-540-24644-2_24
- [6] Paras Jain, Ajay N. Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [7] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2020. Dynamic Tensor Rematerialization. *arXiv preprint abs/2006.09616* (2020).
- [8] Yi Li, Aarti Gupta, and Sharad Malik. 2023. Combined Scheduling, Memory Allocation and Tensor Replacement for Minimizing Off-Chip Data Accesses of DNN Accelerators. *arXiv preprint abs/2311.18246* (2023).
- [9] Vincenzo Liberatore, Martin Farach-Colton, and Ulrich Kremer. 1999. Evaluation of Algorithms for Local Register Allocation. In *Proceedings of the 8th International Conference on Compiler Construction (CC) (Lecture Notes in Computer Science, Vol. 1575)*. Springer, 137–153. doi:10.1007/978-3-540-49051-7_10
- [10] Ghassan Shobaki, Ahmad Al Qutami, and Najm Eldeen Abu-Rmaileh. 2022. Register-Pressure-Aware Instruction Scheduling Using Ant

- Colony Optimization. *ACM Transactions on Architecture and Code Optimization* 19, 4 (2022).
- [11] Ghassan Shobaki, Saeed Shawabkeh, and Najm Eldeen Abu-Rmaileh. 2013. Preallocation Instruction Scheduling with Register Pressure Minimization Using a Combinatorial Optimization Approach. *ACM Transactions on Architecture and Code Optimization* 10, 4 (2013).